

Capacity, Overfitting and Underfitting

Capacity, Overfitting and Underfitting

Given the data set

$$\mathcal{S} = \{x_1, \dots, x_{M+N}\}$$

generated by the same distribution


$$\mathcal{S}_{train} = \{x_1^{train}, \dots, x_N^{train}\}$$

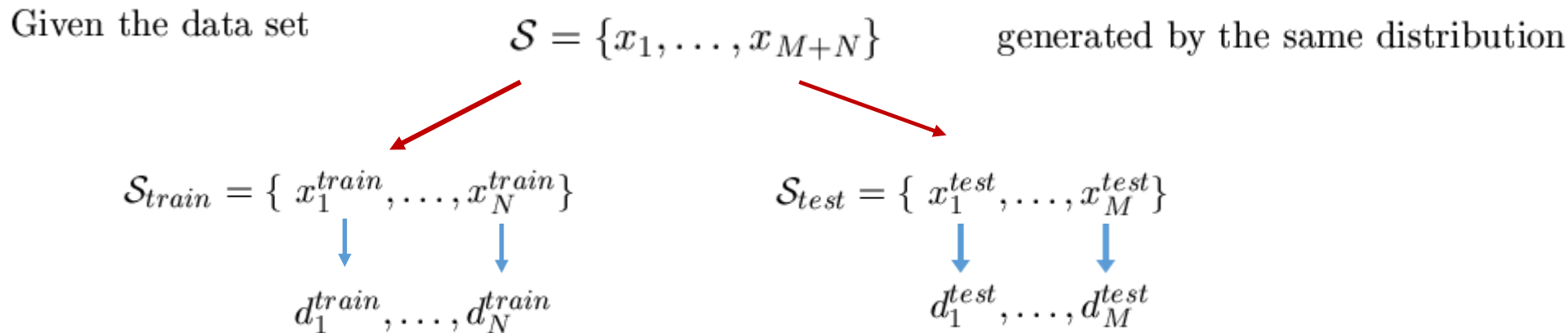

$$\mathcal{S}_{test} = \{x_1^{test}, \dots, x_M^{test}\}$$

Capacity, Overfitting and Underfitting

Given the data set

$$\mathcal{S} = \{x_1, \dots, x_{M+N}\}$$

generated by the same distribution


$$\mathcal{S}_{train} = \{x_1^{train}, \dots, x_N^{train}\}$$

$$d_1^{train}, \dots, d_N^{train}$$

$$\mathcal{S}_{test} = \{x_1^{test}, \dots, x_M^{test}\}$$

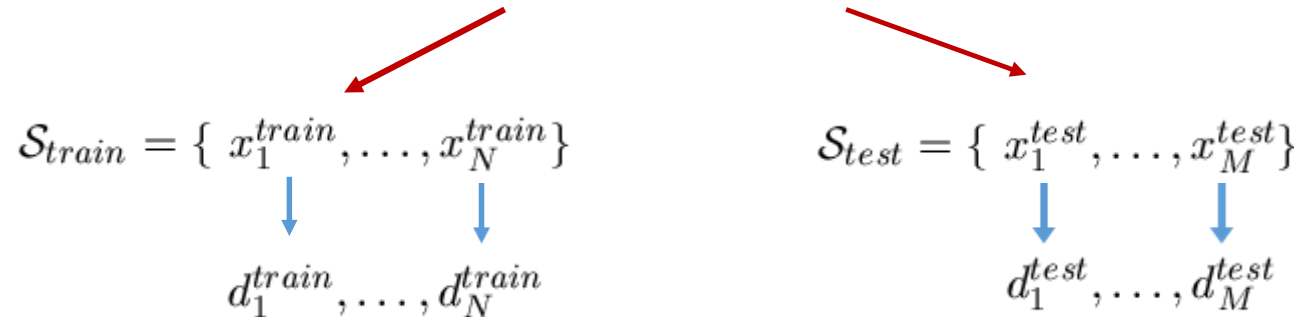
$$d_1^{test}, \dots, d_M^{test}$$

Capacity, Overfitting and Underfitting

Given the data set

$$\mathcal{S} = \{x_1, \dots, x_{M+N}\}$$

generated by the same distribution


$$\mathcal{S}_{train} = \{x_1^{train}, \dots, x_N^{train}\}$$

$$d_1^{train}, \dots, d_N^{train}$$

$$\mathcal{S}_{test} = \{x_1^{test}, \dots, x_M^{test}\}$$

$$d_1^{test}, \dots, d_M^{test}$$

$$o = F(W, x) \quad \leftarrow \text{Network output}$$

cost (loss) function to be minimized

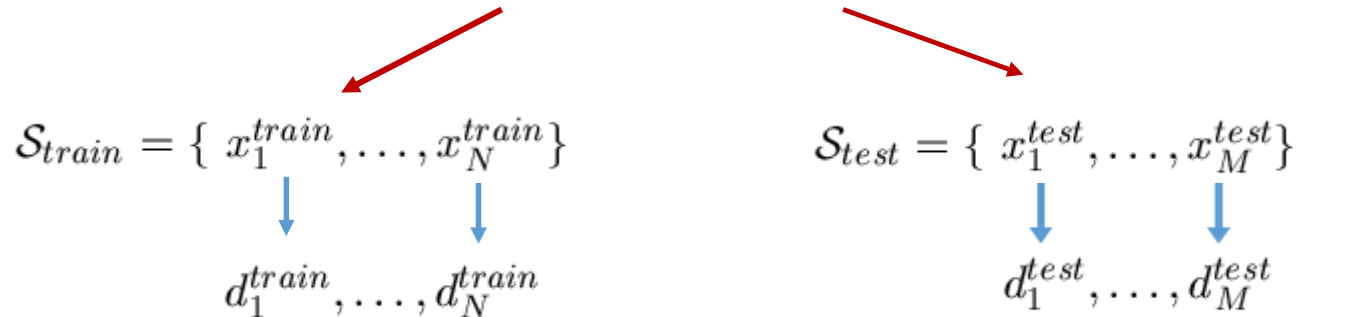

$$E_{train}(W) = \frac{1}{N} \sum_{i=1}^N \|d_i^{train} - o(W, x_i^{train})\|^2$$

Capacity, Overfitting and Underfitting

Given the data set

$$\mathcal{S} = \{x_1, \dots, x_{M+N}\}$$

generated by the same distribution


$$\mathcal{S}_{train} = \{x_1^{train}, \dots, x_N^{train}\}$$

$$d_1^{train}, \dots, d_N^{train}$$

$$\mathcal{S}_{test} = \{x_1^{test}, \dots, x_M^{test}\}$$

$$d_1^{test}, \dots, d_M^{test}$$

$$o = F(W, x) \quad \leftarrow \text{Network output}$$

cost (loss) function to be minimized

$$E_{train}(W) = \frac{1}{N} \sum_{i=1}^N \|d_i^{train} - o(W, x_i^{train})\|^2$$

$$W^* = \operatorname{argmin} E_{train}(W)$$


Capacity, Overfitting and Underfitting

Given the data set

$$\mathcal{S} = \{x_1, \dots, x_{M+N}\}$$

generated by the same distribution

$$\mathcal{S}_{train} = \{x_1^{train}, \dots, x_N^{train}\}$$

$$\mathcal{S}_{test} = \{x_1^{test}, \dots, x_M^{test}\}$$

$$d_1^{train}, \dots, d_N^{train}$$

$$d_1^{test}, \dots, d_M^{test}$$

$$o = F(W, x) \quad \leftarrow \text{Network output}$$

cost (loss) function to be minimized

$$E_{train}(W) = \frac{1}{N} \sum_{i=1}^N \|d_i^{train} - o(W, x_i^{train})\|^2$$

$$W^* = \operatorname{argmin} E_{train}(W)$$

$$E_{train} = E_{train}(W^*)$$

$$= \frac{1}{N} \sum_{i=1}^N \|d_i^{train} - o(W^*, x_i^{train})\|^2$$

Capacity, Overfitting and Underfitting

Given the data set

$$\mathcal{S} = \{x_1, \dots, x_{M+N}\}$$

generated by the same distribution

$$\mathcal{S}_{train} = \{x_1^{train}, \dots, x_N^{train}\}$$

$$\mathcal{S}_{test} = \{x_1^{test}, \dots, x_M^{test}\}$$

%80

$$d_1^{train}, \dots, d_N^{train}$$

$$d_1^{test}, \dots, d_M^{test}$$

%20

$$o = F(W, x) \quad \leftarrow \text{Network output}$$

cost (loss) function to be minimized

$$E_{train}(W) = \frac{1}{N} \sum_{i=1}^N \|d_i^{train} - o(W, x_i^{train})\|^2$$

$$W^* = \operatorname{argmin} E_{train}(W)$$

$$E_{train} = E_{train}(W^*)$$

$$= \frac{1}{N} \sum_{i=1}^N \|d_i^{train} - o(W^*, x_i^{train})\|^2$$

$$E_{test} = \frac{1}{M} \sum_{i=1}^M \|d_i^{test} - o(W^*, x_i^{test})\|^2$$

Typically, when training a machine learning model, we have access to a training set, we can compute some error measure on the training set called the **training error**, and we reduce this training error. So far, what we have described is simply an optimization problem. What separates machine learning from optimization is that we want the **generalization error**, also called the **test error**, to be low as well. The generalization error is defined as the expected value of the error on a new input. Here the expectation is taken across different possible inputs, drawn from the distribution of inputs we expect the system to encounter in practice.

1. Make the training error small.
2. Make the gap between training and test error small.

Underfitting / Overfitting

Bias-Variance dilemma

Capacity is an informal term. It's very close (if not a synonym) for model complexity. It's a way to talk about how complicated a pattern or relationship a model can express. You could expect a model with higher capacity to be able to model more relationships between more variables than a model with a lower capacity.

Having appropriate **CAPACITY** might help to get good generalization.

Task : Function Approximation

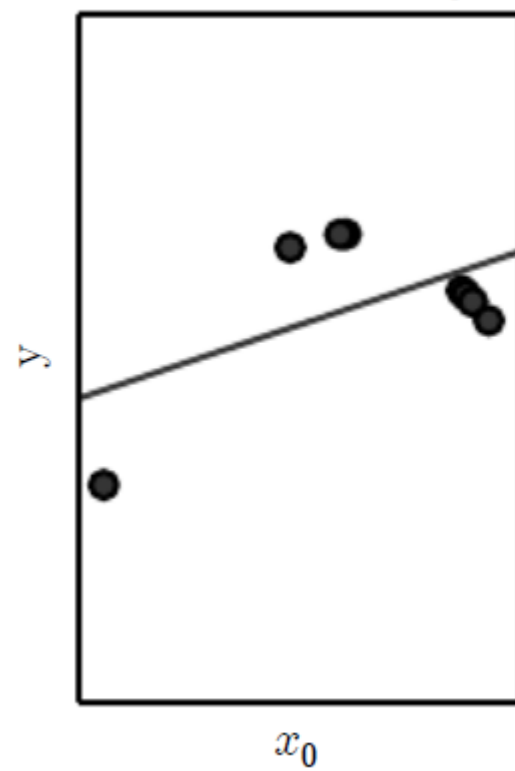
Assume that data comes from a quadratic distribution (**but we don't know that**)

Neuron Model 1 : $\hat{y} = b + wx.$

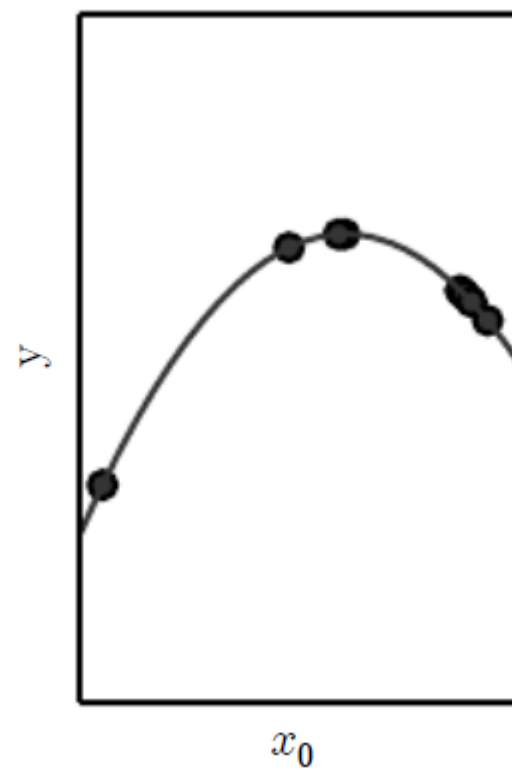
Neuron Model 2 : $\hat{y} = b + w_1x + w_2x^2.$

Neuron Model 3 : $\hat{y} = b + \sum_{i=1}^9 w_i x^i$

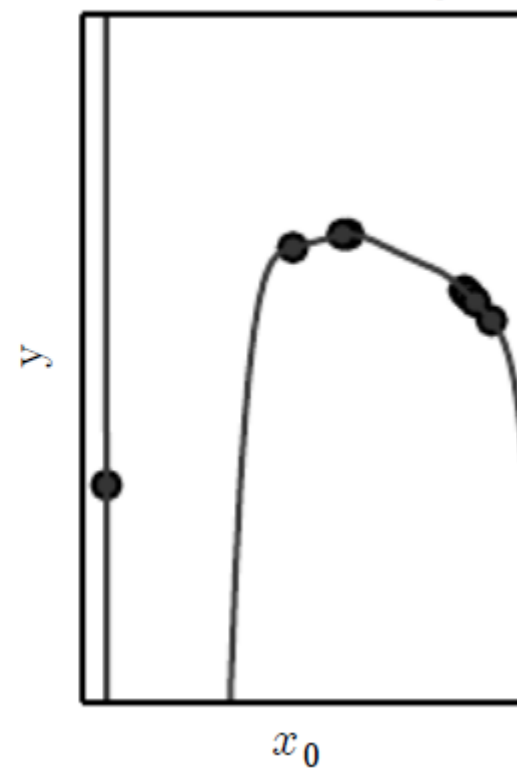
Underfitting



Appropriate capacity



Overfitting



Statistical learning theory provides various means of quantifying model capacity. Among these, the most well-known is the **Vapnik-Chervonenkis dimension**, or VC dimension. The VC dimension measures the capacity of a binary classifier. The VC dimension is defined as being the largest possible value of m for which there exists a training set of m different $\boldsymbol{x}$ points that the classifier can label arbitrarily.

Data set S, Task T : Binary Classification (e.g. 2 classes)

The Model, Hypothesis H : A Mathematical relation of model depending on parameters,
e.g. neuron model ($h(x), f(x)..\)$)

VC dimension of a classification model [edit]

A binary classification model f with some parameter vector θ is said to *shatter* a set of *generally positioned* data points $(x_1, x_2, \dots, x_n)$ if, for every assignment of labels to those points, there exists a θ such that the model f makes no errors when evaluating that set of data points^[citation needed].

The VC dimension of a model f is the maximum number of points that can be arranged so that f shatters them. More formally, it is the maximum cardinal D such that there exists a generally positioned data point set of *cardinality* D can be shattered by f .

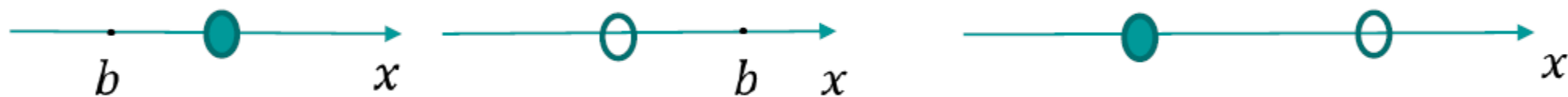
$$VC(h) = \max\{d \mid \text{there exists a set of } d \text{ points that can be shattered by } h\}$$

https://en.wikipedia.org/wiki/Vapnik%E2%80%93Chervonenkis_dimension

■ Example: $H_3 = \{y = \text{sign}(x - b) \mid b \in R\}$

● : +1

○ : -1

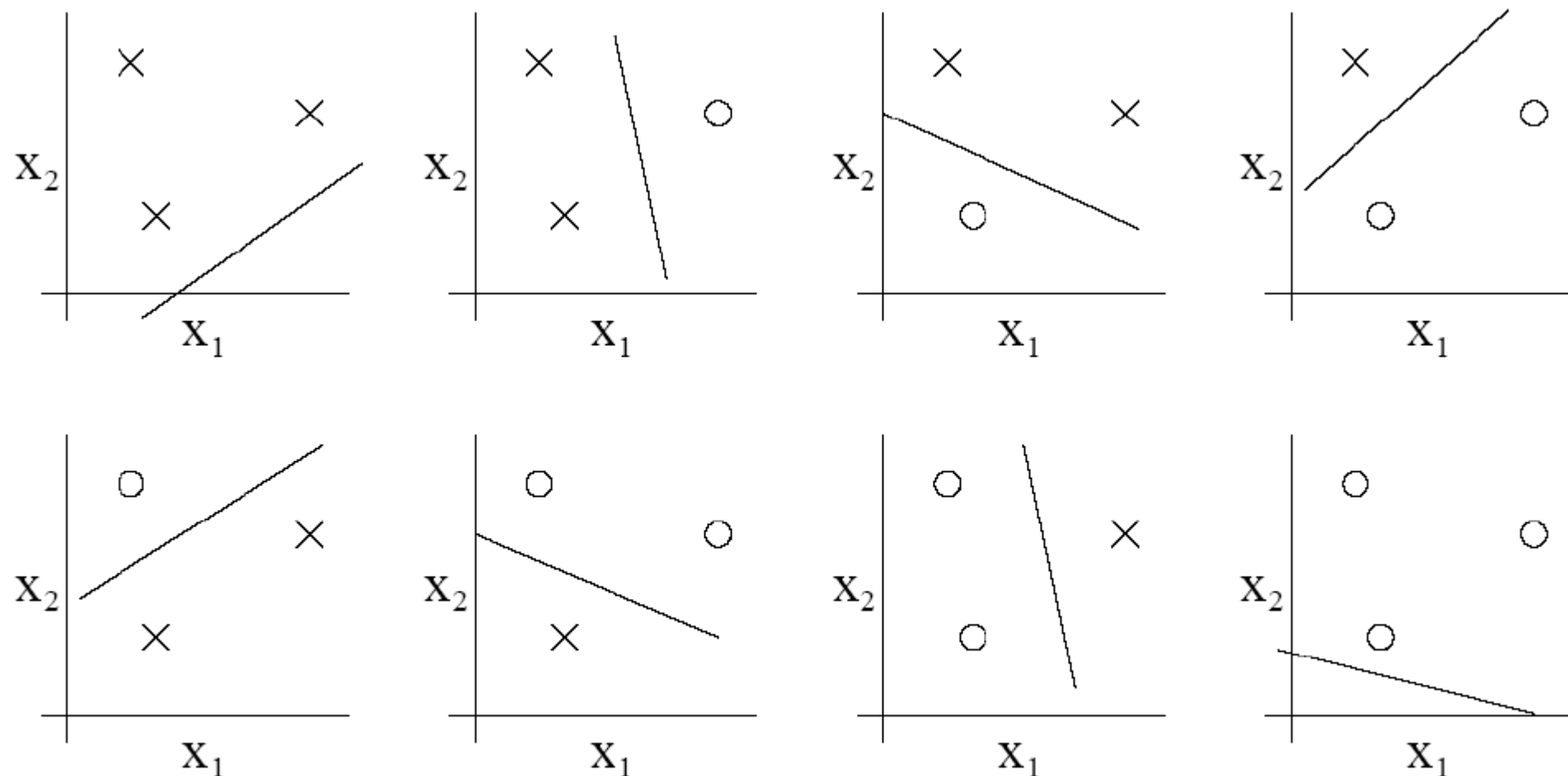


No b value such that the above 2
samples can shattered $\rightarrow d_{VC}(H_3) = 1$

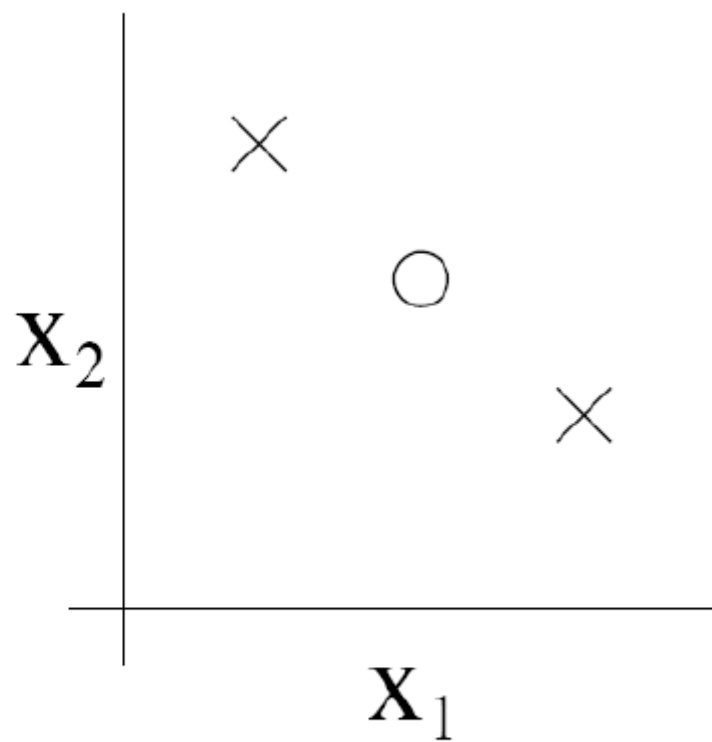
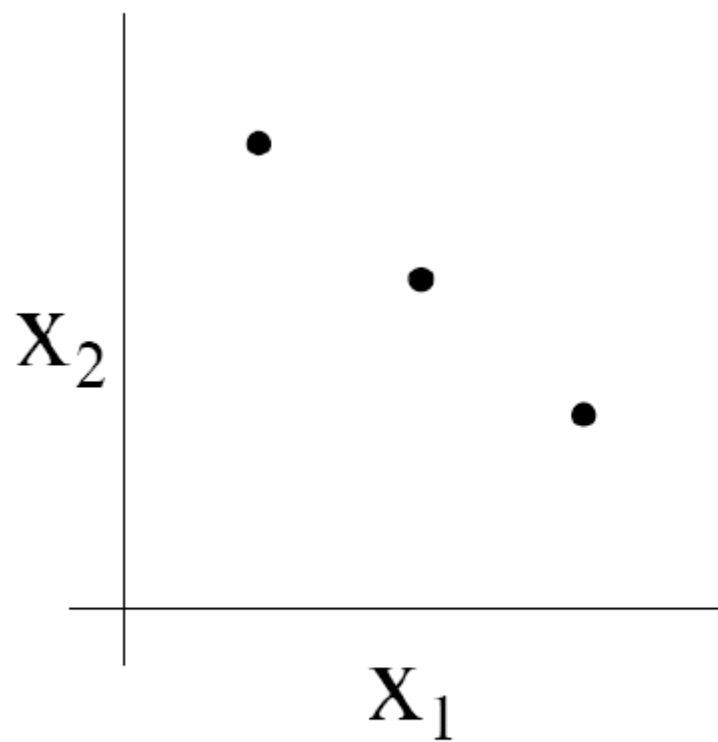
Example: $H_2 = \{y = \text{sign}(w_1x_1 + w_2x_2) \mid w_1, w_2 \in R\}$

$$d_{VC}(H_2) = 2$$

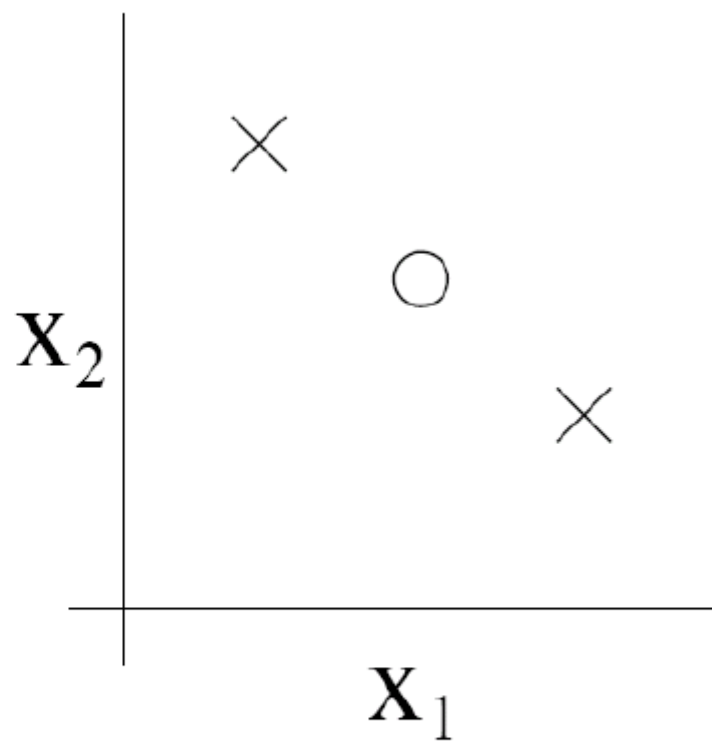
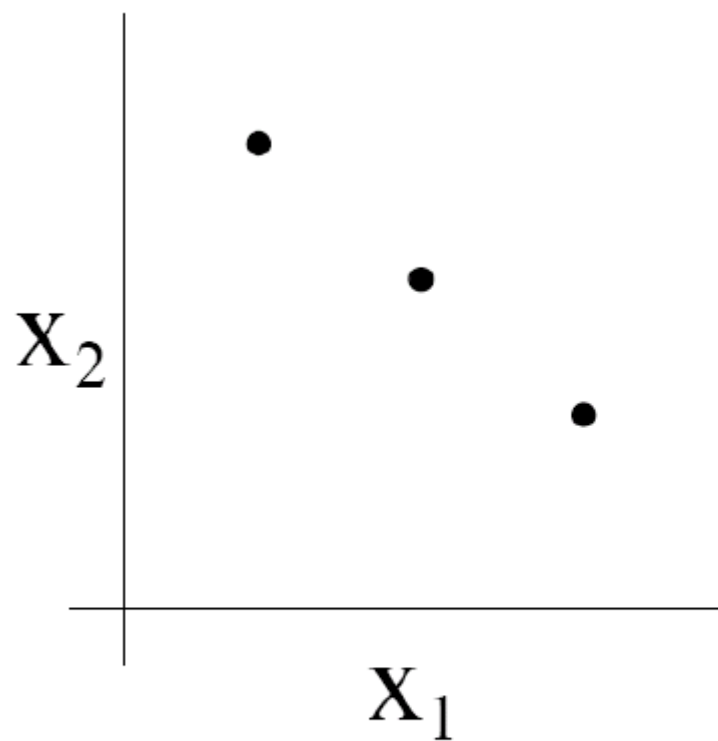
Example: $H_1 = \{y = \text{sign}(w_1x_1 + w_2x_2 + b) \mid w_1, w_2, b \in \mathbb{R}\}$



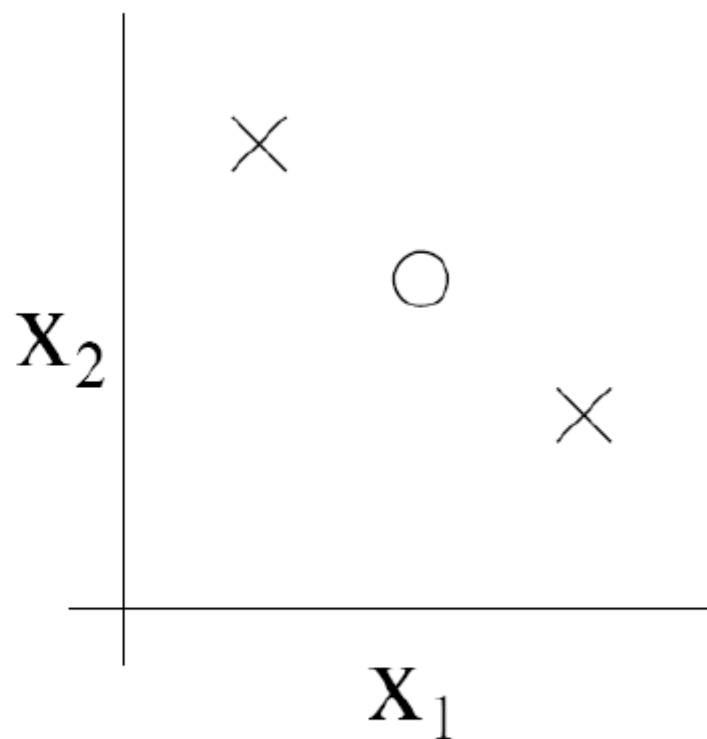
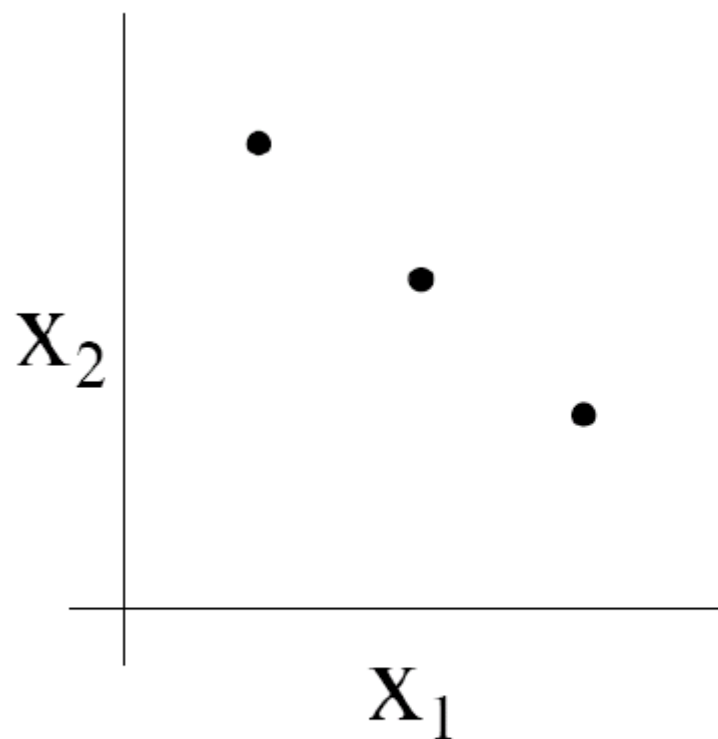
- For any of the eight possible labelings of these points, we can find a linear classifier that obtains "zero training error" on them.



- The VC dimension of H here is 3 even though there may be sets of size 3 that it cannot shatter.



- The VC dimension of H here is 3 even though there may be sets of size 3 that it cannot shatter.
- under the definition of the VC dimension, in order to prove that $VC(H)$ is at least d , we need to show only that there's **at least one set of size d** that H can shatter.



- The VC dimension of H here is 3 even though there may be sets of size 3 that it cannot shatter.
- under the definition of the VC dimension, in order to prove that $VC(H)$ is at least d , we need to show only that there's **at least one set of size d** that H can shatter.
- Moreover, it is possible to show that there is no set of 4 points that this hypothesis class can shatter.

$$\rightarrow d_{VC}(H_1)=3.$$

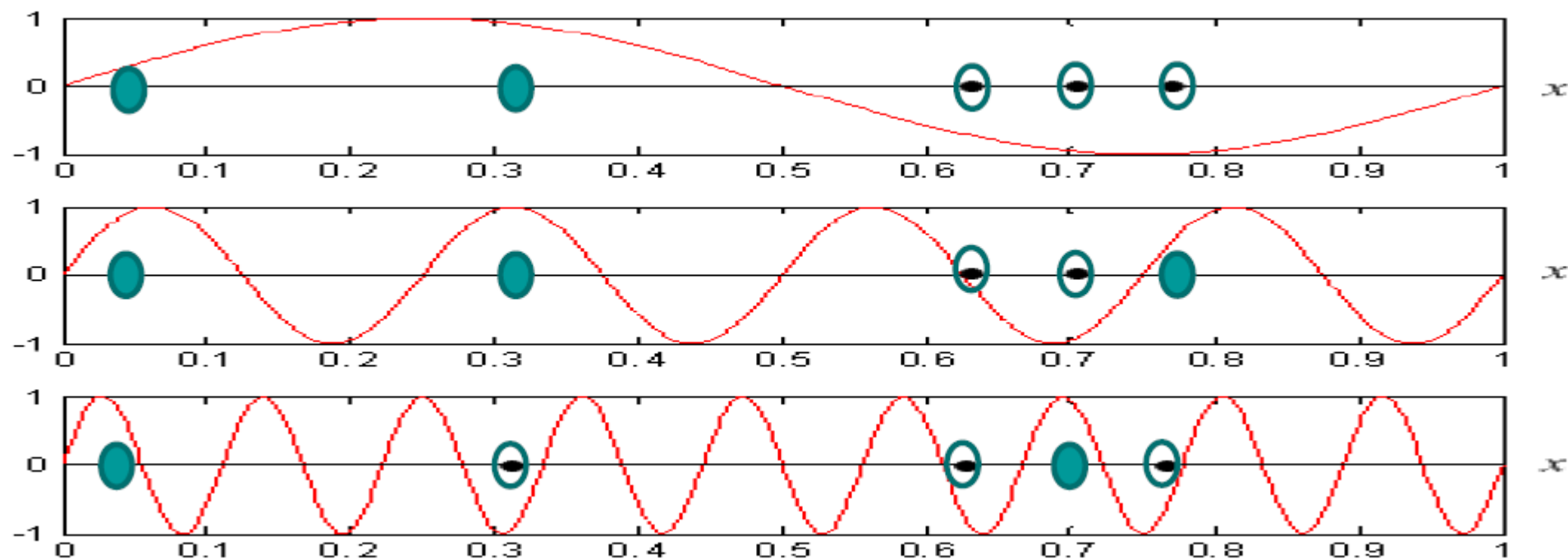
$d_{VC}(H)$ may be equal to or greater than, or less than the number of parameters of the model.

$d_{VC}(H)$ may be equal to or greater than, or less than the number of parameters of the model.

Example: $H_4 = \{y = \text{sign}(\sin(wx)) \mid w \in R\}$

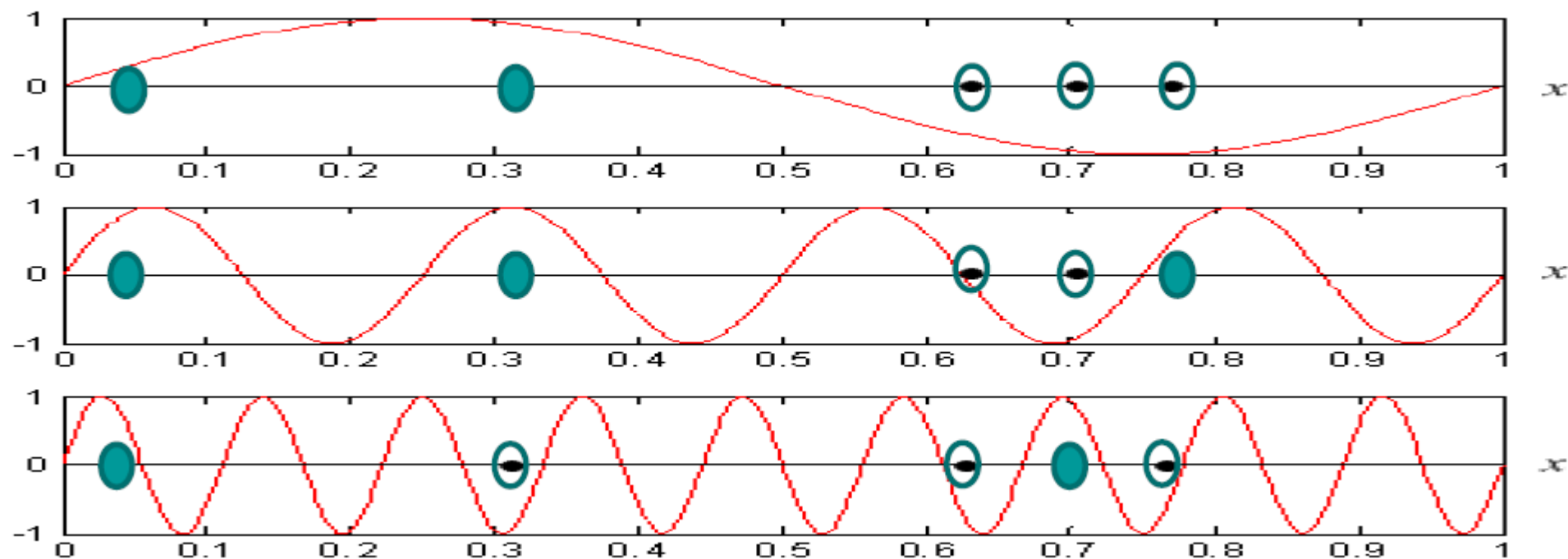
$d_{VC}(H)$ may be equal to or greater than, or less than the number of parameters of the model.

Example: $H_4 = \{y = \text{sign}(\sin(wx)) \mid w \in R\}$



$d_{VC}(H)$ may be equal to or greater than, or less than the number of parameters of the model.

Example: $H_4 = \{y = \text{sign}(\sin(wx)) \mid w \in \mathbb{R}\}$



H_4 can shatter arbitrary number of samples under arbitrary labelling $\rightarrow d_{VC}(H_4) = \infty$.

In statistical learning theory [\[edit \]](#)

The VC dimension can predict a [probabilistic upper bound](#) on the test error of a classification model. Vapnik^[3] proved that the probability of the test error (i.e., risk with 0–1 loss function) distancing from an upper bound (on data that is drawn [i.i.d.](#) from the same distribution as the training set) is given by:

$$\Pr \left(\text{test error} \leq \text{training error} + \sqrt{\frac{1}{N} \left[D \left(\log \left(\frac{2N}{D} \right) + 1 \right) - \log \left(\frac{\eta}{4} \right) \right]} \right) = 1 - \eta,$$

where D is the VC dimension of the classification model, $0 < \eta \leq 1$, and N is the size of the training set (restriction: this formula is valid when $D \ll N$. When D is larger, the test-error may be much higher than the training-error. This is due to [overfitting](#)).

The VC dimension also appears in [sample-complexity bounds](#). A space of binary functions with VC dimension D can be learned with:^{[4]:73}

$$N = \Theta \left(\frac{D + \ln \frac{1}{\delta}}{\varepsilon} \right)^2$$

samples, where ε is the learning error and δ is the failure probability. Thus, the sample-complexity is a linear function of the VC dimension of the hypothesis space.

https://en.wikipedia.org/wiki/Vapnik%E2%80%93Chervonenkis_dimension

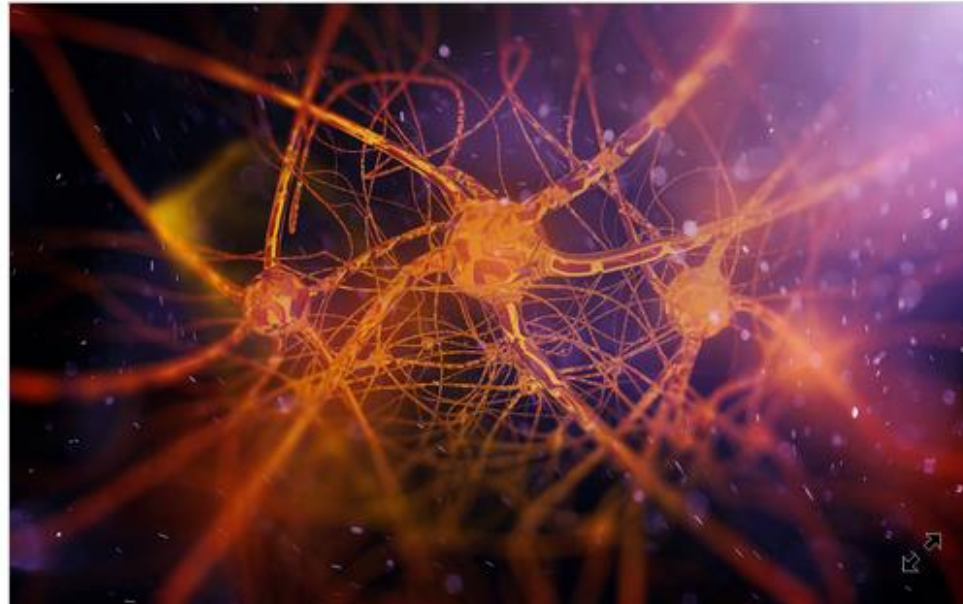
Performance Capacity of a Complex Neural Network

Julia Steinberg

Radix Trading, Chicago, IL, US

July 12, 2023 • *Physics* 16, 108

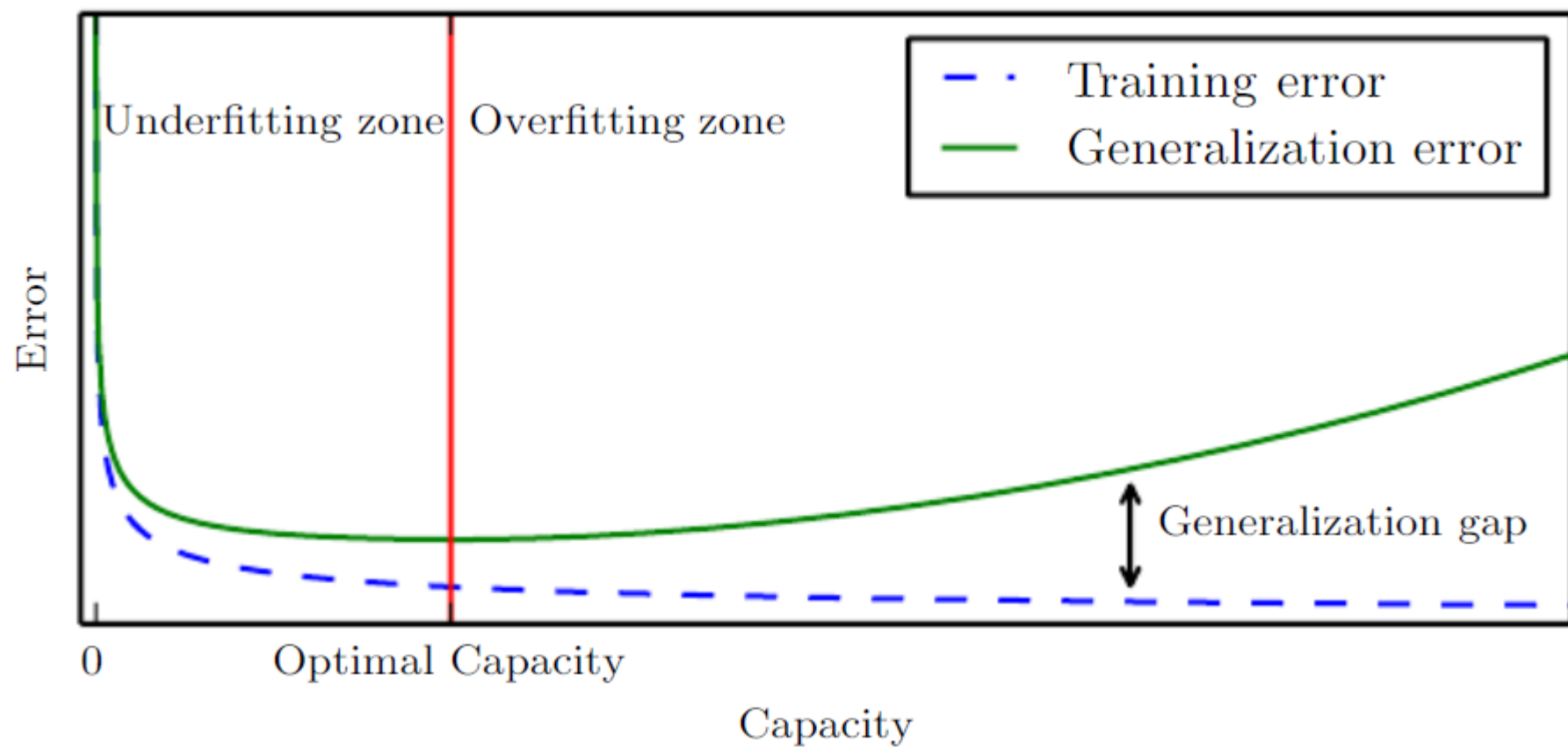
A new theory allows researchers to determine the ability of arbitrarily complex neural networks to perform recognition tasks on data with intricate structure.

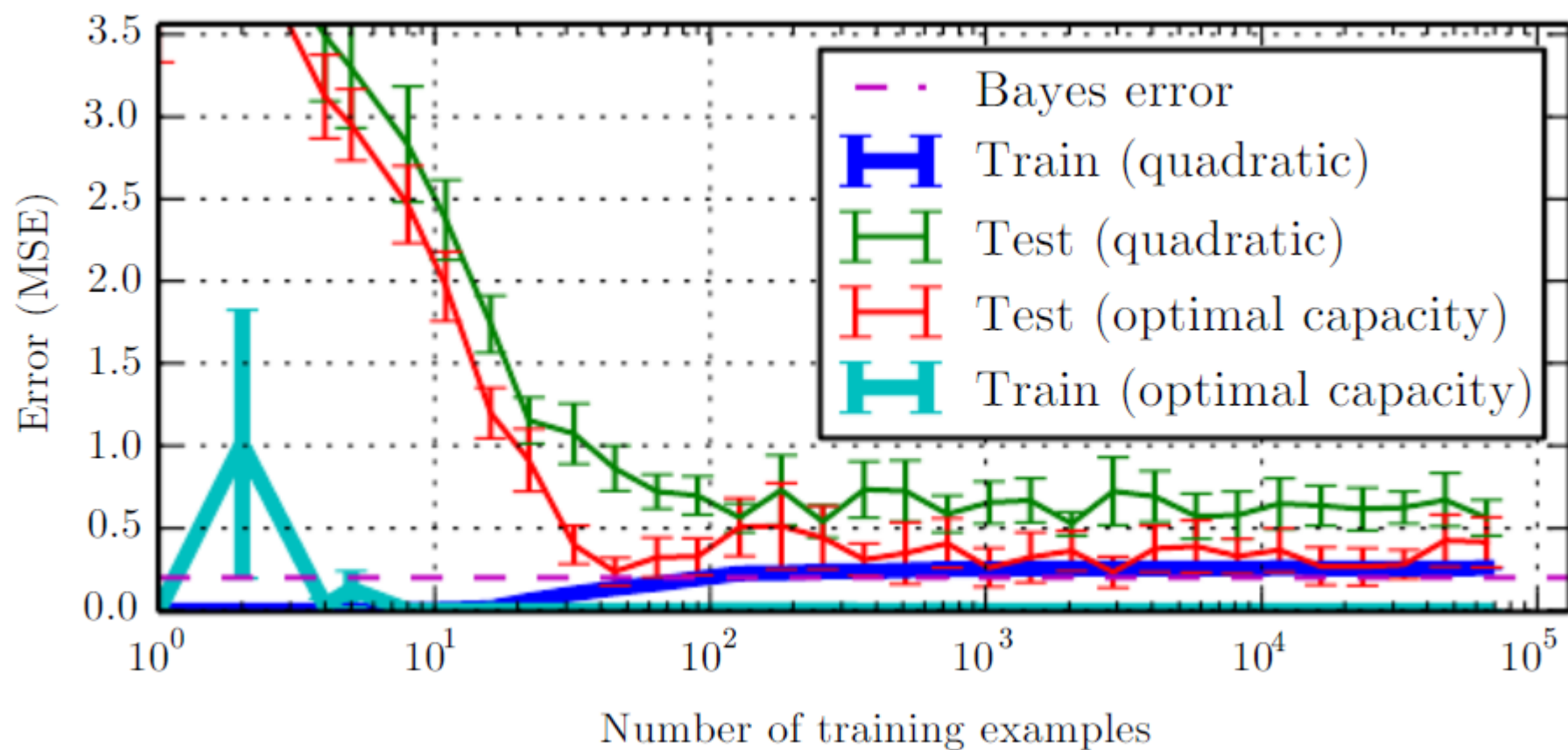


solvod/stock.adobe.com

<https://physics.aps.org/articles/v16/108>

Figure 1: Researchers have developed a method that allows them to determine the performance capacity of a complex neural network.





No free lunch theorem

IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION, VOL. 1, NO. 1, APRIL 1997

No Free Lunch Theorems for Optimization

David H. Wolpert and William G. Macready

Theorem 1: For any algorithms a_1 and a_2 , at iteration step m

$$\sum_f P(d_m^y \mid f, m, a_1) = \sum_f P(d_m^y \mid f, m, a_2),$$

where d_m^y denotes the ordered set of size m of the cost values y associated to input values $x \in X$, $f : X \rightarrow Y$ is the function being optimized and $P(d_m^y \mid f, m, a)$ is the **conditional probability** of obtaining a given sequence of cost values from algorithm a run m times on function f .

https://en.wikipedia.org/wiki/No_free_lunch_theorem#cite_note-WM97-5